AI-based 3D surface area estimation from CT-like voxel data

PHC-351 Course Project 51

Manas Jain

Enrolment no. 24123021

Outline

1. Introduction: why surface area, what a CT scan gives us, how area is measured
2. Problem and objectives
3. Theory: why each classical method is wrong, and by how much
4. Previous work
5. Method: data, exact reference areas, models, training
6. Live demo
7. Results: accuracy, noise, object size, grid size, unseen shapes, failure case
8. Conclusion, limitations and next steps

Why surface area matters

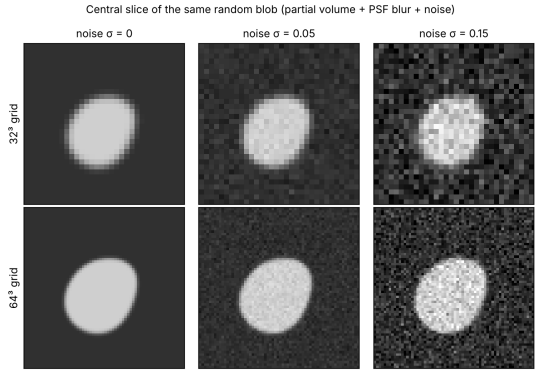
Surface area is measured from X-ray CT scans in many fields:

- **Catalysts**: reactions happen on the surface, so more area means more activity.
- **Filters and rocks**: the pore surface controls how fluid flows through the material.
- **3D-printed lattices**: the area shows whether a part matches its design.

Question of this project

How wrong is the surface area we measure from a CT scan, and can a learned model correct it?

What a CT scan gives us



- A 3D grid of small cubes (**voxels**).
- Each voxel stores one grey value: dark outside, bright inside.
- No surface is stored anywhere: it has to be reconstructed.

Same blob at 32^3 and 64^3, noise $\sigma_n = 0,\ 0.05,\ 0.15$.

Three things that blur the picture

1. **Partial volume**: a voxel's value is the fraction of it inside the object.

$$v = \frac{\text{volume of voxel inside the object}}{\text{volume of voxel}}$$

   A voxel half inside comes out half grey.
2. **Scanner blur** (point-spread function): each value is mixed with its neighbours.
3. **Noise**: random fluctuations added to every voxel.

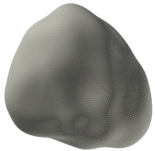
Consequence

The surface lies between voxel centres. Every method that reconstructs it adds some error.

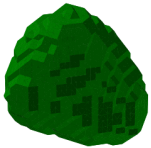
Two standard ways to measure area

Rough blob on a 32^3 grid, no noise: staircase vs interpolated surfaces

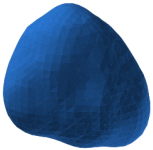
reference surface
A = 1085 voxel²



MC on binary volume
+6.6%



MC on greyscale volume
-2.0%



1. Count voxel faces

Threshold at 0.5, count the faces on the outside of the inside voxels.

2. Marching cubes

Fit a triangle mesh through the grid and add up the triangle areas.

How marching cubes works

- Look at each $2 \times 2 \times 2$ cell of voxels.
- Wherever the value crosses the iso-level 0.5 along an edge, place a vertex.
- Connect the vertices into triangles using a lookup table of cell patterns.

Where on the edge? Between voxels with values v_a and v_b:

$$t = \frac{0.5 - v_a}{v_b - v_a}$$

Binary volume

Values are 0 or 1, so $t = 0.5$ always: vertices only at edge midpoints.

Greyscale volume

t can take any value, so the surface can sit anywhere between voxels.

Problem and objectives

Problem

Given a blurred, noisy voxel volume of one object, estimate the area of its true surface.

Objectives

1. Shapes with a true area known to better than 1 part in 10,000.
2. Measure the error of voxel faces and three marching-cubes variants.
3. Learn a correction (3D CNN, PointNet, Random Forest) and compare it with a lookup table.
4. Test on shape families never seen in training.
5. Say when a learned correction is worth using.

Theory 1: why voxel faces measure a staircase

- A flat patch of area A with unit normal $\mathbf{n} = (n_x, n_y, n_z)$.
- Its voxel version is a staircase of faces pointing along x, y and z.
- The x-faces add up to the shadow of the patch on the yz plane: $A\,|n_x|$.
- Same for y and z, so

$$\frac{A_{\text{faces}}}{A} = r(\mathbf{n}) = |n_x| + |n_y| + |n_z|$$

Range

$r = 1$ only when the patch is aligned with the grid.

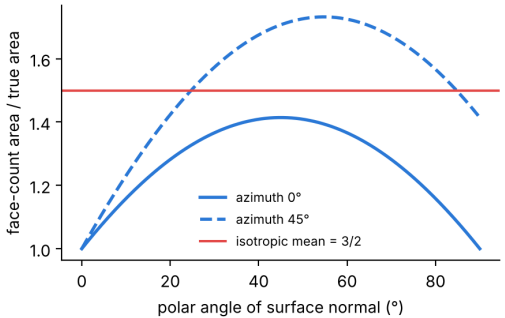
$r = \sqrt{3} \approx 1.73$ on the main diagonal.

Theory 1: on average 50% too high, at any resolution

- For random orientations, each $|n_i|$ is uniform on $[0, 1]$, so $\langle |n_i| \rangle = 1/2$.
- Therefore $\langle r \rangle = 3/2$: a $+50\%$ error.
- r does not contain the voxel size: smaller voxels give finer steps, not flatter ones.

Measured, clean sphere at 128^3

Voxel faces: $+50.0\%$



Theory 2: binary marching cubes never converges

- On a binary volume every vertex is at an edge midpoint.
- So only a few triangle orientations are possible, and a tilted plane becomes a fine zig-zag with extra area.
- Smaller voxels make the zig-zag finer but not flatter (Kenmochi and Klette 2000).

Clean spheres	16^3	32^3	64^3	128^3
Voxel faces	+49.0%	+49.1%	+49.7%	+50.0%
Binary MC	+7.3%	+8.1%	+8.6%	+8.8%

Both errors stay put as the grid is refined; the binary one even grows slightly.

Theory 3: greyscale marching cubes and curvature

- Greyscale values let the vertices sit between voxels. For a flat surface and a symmetric blur, the 0.5 level lies exactly on the true surface.
- For a curved surface it does not. Blurring spreads the object's brightness outwards over a larger volume, so the half-brightness surface sits slightly **inside** the true one.
- The more sharply the surface bends, the bigger the shift.

Result

Greyscale marching cubes is slightly too small on curved surfaces, especially for small objects.

Theory 3: a prediction we can test

Gaussian blur is diffusion, so the 0.5 level moves inwards by

$$\delta \approx \sigma^2 H, \qquad H = \text{mean curvature}.$$

For a sphere of radius R, $H = 1/R$, and

$$\frac{\Delta A}{A} \approx -\frac{2\sigma^2}{R^2}.$$

Our blur: voxel box $1/12$ + scanner 0.6^2 gives $\sigma^2 \approx 0.44$ voxel^2.

Clean spheres, greyscale MC

Grid	R	Predicted	Measured
16^3	5.4	-3.04%	-3.65%
32^3	12.6	-0.56%	-0.62%
64^3	27.0	-0.12%	-0.05%
128^3	55.8	-0.03%	$+0.07\%$

Theory 4: noise creates fake specks

- A background voxel crosses the 0.5 threshold by chance with probability

$$p = Q\left(\frac{0.5}{\sigma_n}\right) \quad (Q = \text{Gaussian tail}).$$

- At $\sigma_n = 0.15$: $p \approx 4.3 \times 10^{-4}$, so about **112 specks** on a 64^3 grid and 14 on a 32^3 grid.
- Each speck is a tiny closed surface that adds area.
- The Euler number (number of separate pieces, roughly) counts them: we measure about 100 to 125 at 64^3.
- At $\sigma_n = 0.05$: $p = Q(10)$, effectively zero.

Theory 4: rough surfaces, and why smoothing is no cure

- Noise also roughens the real surface, and a rough surface always has more area than a smooth one.
- Smoothing the volume first removes most of the noise, but:
  - it increases the blur (σ^2 from 0.44 to 1.44), so the curvature error grows about $3\times$;
  - it erases structures thinner than about two voxels completely.

MC after smoothing, test MAPE	32^3	64^3
All shapes	13.78%	3.04%

Theory: two errors pulling in opposite directions

Curvature: area too small

Strongest for small objects and coarse grids.

Noise: area too big

Strongest for noisy scans and fine grids.

Consequence

The size of each effect depends on the object and the scan, so no single correction factor can fix both. The correction has to be worked out object by object, which is what a learned model can do.

Previous work 1: digital geometry

- **Mesh vs voxels**: triangulated surfaces are much more accurate than voxel-face surfaces (Nyström et al. 2002).
- **Better fixed weights**: optimal weights for each local $2 \times 2 \times 2$ voxel pattern (Mullikin and Verbeek 1993; Windreich et al. 2003; Lindblad 2005).
- **Convergence**:
  - binary marching cubes does not converge to the true area, even for spheres (Kenmochi and Klette 2000);
  - no local estimator converges on convex polytopes (Svane 2013).

These results explain the fixed errors of the binary methods, and point towards using greyscale information.

Previous work 2: learning on 3D data, and the gap

- 3D CNNs on voxel grids (VoxNet, 2015) and PointNet on point sets (2017) are standard tools for 3D shape analysis.
- DeePore (2020): a CNN predicts properties of porous media from micro-CT images, but without an exact ground truth for surface area.
- Random Forests on hand-picked features remain a strong baseline for small problems.

Gap addressed here

Measure, against an exact reference, how much a learned model really reduces the error of marching cubes, separately by resolution, noise and unseen shape family, and compare it with a simple baseline.

Method: seven shape families

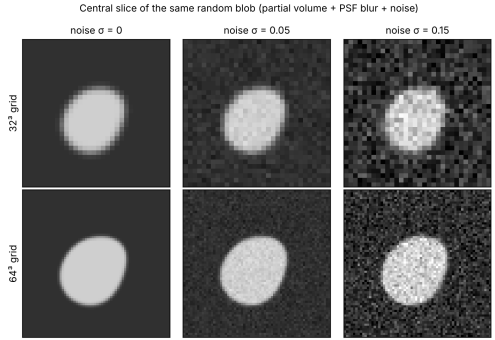
Family	Role	Shapes	Parameters
Sphere	train	150	radius 1
Ellipsoid	train	350	axes 1, $b, c \in [0.35, 1]$
Cylinder (capped)	train	300	radius 0.3 to 0.85
Torus	train	300	tube/ring ratio 0.2 to 0.6
Smooth blob	train	900	6 low-frequency waves on a sphere
Superellipsoid	held out	200	pointed tips or box-like rounded edges
Rough blob	held out	200	smooth blob + 10 high-frequency waves

The two held-out families have features the training shapes never have: sharp edges and fine roughness.

Method: making CT-like volumes

1. Scale (diameter 9.8 to 60 voxels), rotate randomly, shift by a sub-voxel offset.
2. **Partial volume**: $5 \times 5 \times 5$ samples per voxel near the surface.
3. **Blur**: Gaussian, $\sigma = 0.6$ voxel.
4. **Noise**: Gaussian, $\sigma_n \in \{0, 0.05, 0.15\}$.
5. Store as 8-bit values.

Every shape at 32^3 and 64^3 and all three noise levels: 6 volumes per shape.



Method: the exact reference area

- **Exact formulas**: sphere, cylinder, torus, and ellipsoid (using elliptic integrals).
- **Blobs and superellipsoids**: a very fine triangle mesh at two levels of detail (81,920 and 327,680 triangles), combined to cancel the main error:

$$A_{\text{ref}} = \frac{1}{3}\left(4A_{\text{fine}} - A_{\text{coarse}}\right)$$

- Remaining error below 0.01% for every shape; 14 of 14 checks pass.

Why not marching cubes on a very fine grid?

Binary marching cubes never converges, so it would put its own error into the ground truth.

Method: the dataset

2,400	shapes
14,400	CT-like volumes
1,400	training shapes
299	validation shapes
301	test shapes
400	held-out shapes (2 families)

- Split **by shape**, 70/15/15 within each training family, so all six volumes of a shape stay together.
- No shape appears in both training and testing.
- Fixed random seed (51): the whole dataset can be regenerated exactly.

Method: what the models learn

The models do not predict the area directly. They predict a **correction** to the greyscale marching-cubes area:

$$y = \log \frac{A_{\text{true}}}{A_{\text{MC}}}, \qquad \hat{A} = A_{\text{MC}}\, e^{\hat{y}}$$

- Marching cubes is already close, so the model only has to learn the small remaining error.
- y does not depend on the object's size or orientation.
- So each training volume can be rotated and flipped into **48 versions** (the symmetries of a cube) without changing the true area.

Method: the baselines

Lookup table

One fixed correction per resolution and noise level (the median from training). It is even given the true noise level. A learned model must beat it to show it does more than apply an average.

Random Forest (400 trees) on 10 simple measurements

Marching-cubes area relative to the grid; ratios of the face, binary and smoothed areas to it; compactness; two aspect ratios; Euler number; noise estimate; grid size N.

Method: the 3D CNN

- Input: the whole N^3 greyscale volume.
- 5 convolution blocks with 16, 32, 64, 128 and 256 channels (3^3 kernels, batch norm, ReLU, max pooling).
- Global average and max pooling, so the same network works at 32^3 and 64^3.
- A small head (128 units) outputs $\hat{y}$.
- 3.59 million parameters.

Why a CNN?

It can see everything at once: object size, curvature, thin parts and noise specks.

Method: PointNet

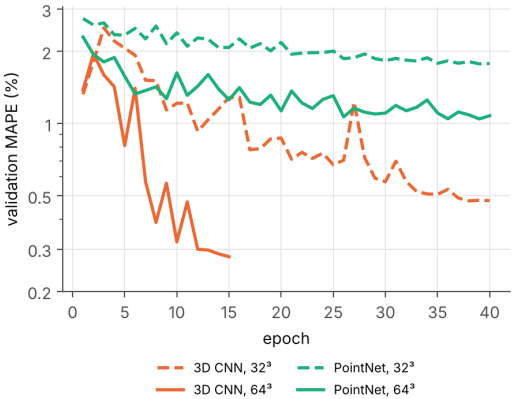
- Input: 2,048 points sampled on the greyscale marching-cubes surface.
- A shared network maps each point to 1,024 features; max pooling combines them.
- A small head (512, 128 units) outputs $\hat{y}$.
- Random rotations during training instead of an alignment network.
- 0.74 million parameters.

Why test it?

A point cloud is a natural description of a surface, and much smaller than a volume.

Method: training

- One model per grid size.
- Huber loss on $y/0.05$ (quadratic below a 5% error).
- AdamW, one-cycle learning rate (peak 10^{-3}), batch 32.
- Up to 40 epochs; 15 for the 64^3 CNN.
- Early stopping on the validation error.
- Apple-silicon laptop: about 60 s per epoch at 32^3, 7 min at 64^3.



Live demo

```
$ python src/demo.py --example blob --N 64 --noise 0.15
synthetic blob, 64^3 grid, noise 0.15, diameter 55.7 voxels
Voxel-face counting          +83.72%
Marching cubes (binary)      +23.37%
Marching cubes (greyscale)    +9.25%
3D CNN correction             +0.24%

$ python src/demo.py --example ellipsoid --N 32 --noise 0.05
synthetic ellipsoid, 32^3 grid, noise 0.05, diameter 10.5 voxels
Marching cubes (greyscale)    -7.52%
3D CNN correction             -0.96%
```

Noise pushes greyscale MC up, curvature pulls it down; the same CNN corrects both.

Results: test shapes never seen in training

Average error (MAPE)	32^3	64^3
Voxel faces	56.71%	65.54%
MC (binary)	10.38%	14.49%
MC (greyscale)	4.78%	3.52%
MC (smoothed)	13.78%	3.04%
MC + lookup	2.97%	0.79%
PointNet	1.98%	1.04%
Random Forest	0.72%	**0.18%**
3D CNN	**0.68%**	0.26%

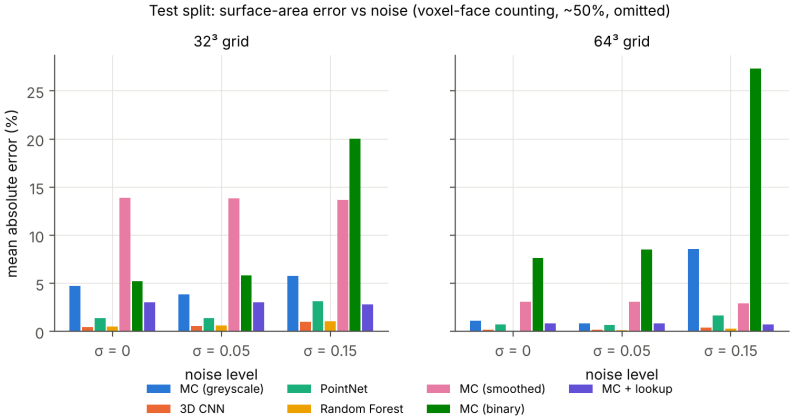
- The CNN cuts the greyscale MC error 7 to 13 times.
- Its average bias is within $\pm 0.1\%$.
- The Random Forest does just as well, and best at 64^3.
- PointNet is clearly weaker.

Results: what the Random Forest learned

Measurement	Importance
Euler number (counts specks)	0.29
Noise estimate	0.26
Smoothed / raw MC area	0.22
Faces / MC area	0.08
Everything else	0.15

- The top two measure exactly the two noise effects from the theory.
- The smoothed-to-raw ratio measures how much the surface responds to smoothing: a stand-in for curvature and thin parts.
- Physics-based features carry most of the information.

Results: robust to noise



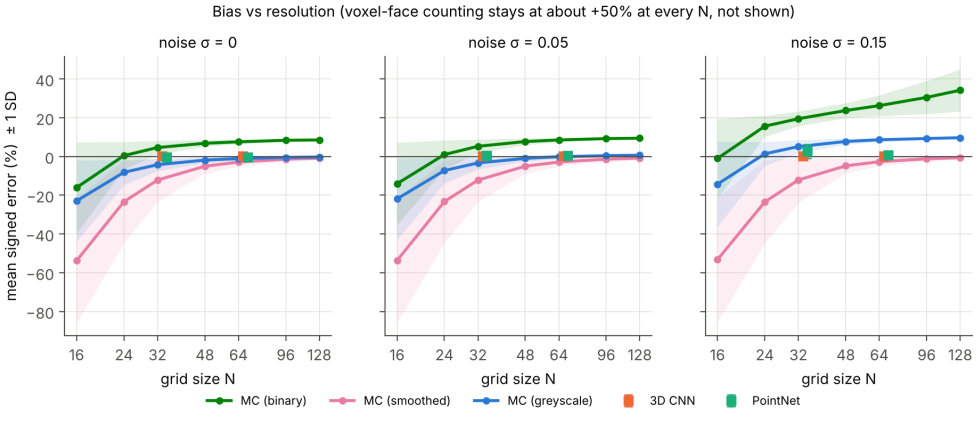
- 64^3: greyscale MC goes from 1.13% (clean) to 8.56% (highest noise).
- The CNN only goes from 0.20% to 0.40%; the Random Forest from 0.09% to 0.30%.

Results: object size matters more than grid size

Object diameter (voxels)	< 16	16–24	24–32	32–48	48–64
MC (greyscale)	6.43%	3.89%	3.57%	3.45%	3.44%
MC + lookup	4.76%	1.78%	1.48%	0.67%	0.58%
Random Forest	1.41%	**0.39%**	**0.22%**	**0.13%**	**0.08%**
PointNet	2.14%	1.57%	1.54%	0.81%	1.48%
3D CNN	**1.24%**	**0.39%**	0.28%	0.22%	0.22%

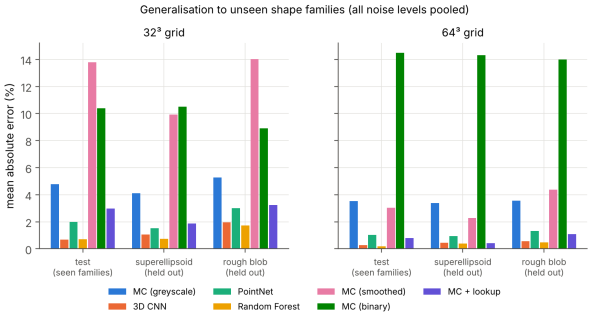
- Small objects: the CNN is best; this is where seeing the whole image pays off.
- Large objects: the Random Forest is best, down to 0.08%.

Results: the theory holds across grid sizes



- Binary MC: clean $+0.5\% \to +8.5\%$ from 24^3 to 128^3; with noise $+16\% \to +34\%$.
- Greyscale MC: -22.9% at 16^3 (curvature), $+9.7\%$ at 128^3 with noise.
- Learned models (squares) stay near zero at the two sizes they were trained on.

Results: shape families never seen in training



- **Superellipsoids** (sharp edges): CNN 1.05% / 0.45%, Random Forest 0.73% / 0.39%, greyscale MC 4.10% / 3.38%.
- **Rough blobs**: hardest case, CNN 1.96% / 0.55%. Biased low: roughness smaller than a voxel is not in the image.
- Errors are 1.0 to 2.2 times the test-set errors: a moderate gap.

Results: a failure case

- The 32^3 errors have a long tail: the CNN's average error is 0.68%, but its spread is 3.1%.
- Most of that comes from one torus with a tube only **1.77 voxels** thick.
- After blurring, most of the tube never reaches the 0.5 level: greyscale MC finds only **42 of 142** units of area; the smoothed volume finds none.
- Every model is off by 47% to 66%.

Lesson

A correction can fix a biased surface, but not a surface that is missing from the mesh.

Which method should you use?

Situation	Recommendation
Any case	Do not use voxel faces or binary MC for absolute area.
Large, clean objects	Greyscale MC is already within about 1%.
Noisy scans	A learned correction brings the error to about 1% or less.
Objects under 16 voxels	Use the 3D CNN: 1.24%, against 6.43% for greyscale MC.
Thinner than 2 voxels	No method tested here is reliable.

Conclusion

1. The classical errors are not random: they come from geometry, curvature and noise, and simple theory predicts them.
2. The best corrections reach 0.68% (32^3) and 0.18% (64^3) on unseen test shapes, from 4.78% and 3.52%.
3. They still work on shape families never seen in training, with errors below 2%.
4. A simple Random Forest does as well as the 3D CNN in most cases.

Limitations

- **Simplified CT physics**: noise added to the image, not to the X-ray projections; no artefacts such as beam hardening or rings.
- **One object per volume**: real materials have many touching structures.
- **Fixed scan settings**: two grid sizes, three noise levels, fixed blur.
- **Sub-voxel detail**: on rough surfaces the correction is a learned guess, not a measurement.
- **Simple shapes, no tuning**: each model trained once; the 64^3 CNN only 15 epochs.
- **No real scan** with a known surface area yet.

Next steps

1. **Realistic CT simulation**: simulate the X-ray projections, Poisson noise and the reconstruction, instead of adding noise to the image.
2. **Many objects per volume**: foams, porous rocks and printed lattices.
3. **Real data**: scan an object whose surface area is known and check the correction.
4. **More training**: longer training at 64^3, and more shape types.



Thank you

Manas Jain | 24123021